Cross-Validation: Linear vs Polynomial Regression on Auto Data

Aileen Tao

Table of contents

Plan 1

Methods and replication 2

Results 5

Conclusion 8

Plan

In this project, I want to understand how MPG relates to horsepower in the **ISLR Auto dataset**. A simple way to model this relationship is with a straight line, but the relationship might also be curved. If the pattern bends, a linear model may miss some structure in the data, while a polynomial model could capture it better.

To investigate this, I compare two models that predict **mpg using horsepower**:

- **Linear: mpg ~ horsepower**
- **Polynomial (degree 3): mpg ~ poly(horsepower, 3)**

I fixed the polynomial degree at 3 ahead of time instead of searching for the "best" degree using the full dataset. This avoids tuning the model in a way that could bias the evaluation.

To compare the models fairly, I use 10-fold cross-validation. The data are split into 10 folds, the model is trained on 9 folds, and tested on the remaining fold. This process is repeated so every observation is used for validation once.

Because the result of a single split can depend on random chance, I repeat the entire 10-fold CV procedure 50 times using different random partitions of the data. For each run I compute the mean validation MSE, and then compare the averages across the 50 runs.

I also compute the training MSE using the full dataset, mainly to illustrate how training error can look overly optimistic compared to cross-validation.

Methods and replication

All analysis steps are written so the results can be reproduced exactly.

First, I load the required packages and set a seed so the random sampling is consistent.

```
library(ISLR)
library(tidyverse)

set.seed(2026)
K <- 10L
n_rep <- 50L
poly_degree <- 3L
```

Then I load the Auto dataset from the ISLR package and inspect its structure. The dataset contains 392 cars and 9 variables, including mpg, horsepower, weight, cylinders, and several other attributes.

```
data(Auto, package = "ISLR")
glimpse(Auto)
```

```
Rows: 392
Columns: 9
$ mpg          <dbl> 18, 15, 18, 16, 17, 15, 14, 14, 14, 15, 15, 14, 15, 14, 2~
$ cylinders    <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 4, 6, 6, 6, 4, ~
$ displacement <dbl> 307, 350, 318, 304, 302, 429, 454, 440, 455, 390, 383, 34~
$ horsepower   <dbl> 130, 165, 150, 150, 140, 198, 220, 215, 225, 190, 170, 16~
$ weight       <dbl> 3504, 3693, 3436, 3433, 3449, 4341, 4354, 4312, 4425, 385~
$ acceleration <dbl> 12.0, 11.5, 11.0, 12.0, 10.5, 10.0, 9.0, 8.5, 10.0, 8.5, ~
$ year         <dbl> 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 7~
$ origin       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 3, ~
$ name         <fct> chevrolet chevelle malibu, buick skylark 320, plymouth sa~
```

Next, I perform a small amount of cleaning. If horsepower appears as a character variable, I convert it to numeric and treat any problematic values as missing. Rows with missing values are then removed.

```
data(Auto, package = "ISLR")

if (any(sapply(Auto, is.character))) {
  Auto[] <- lapply(Auto, function(v) {
    if (is.character(v)) type.convert(v, as.is = TRUE) else v
  })
}

if ("horsepower" %in% names(Auto)) {
  hp <- Auto$horsepower
  if (is.character(hp)) {
    Auto$horsepower <- suppressWarnings(as.numeric(hp))
  }
}

Auto <- na.omit(Auto)
n <- nrow(Auto)
n
```

```
[1] 392
```

After cleaning, the dataset contains 392 observations, with:

- **mpg** as the response variable
- **horsepower** as the predictor

I use 10-fold cross-validation and repeat the procedure 50 times. In each repetition, the data are randomly divided into 10 folds. For each fold, the models are trained on the other nine folds and evaluated on the remaining fold. I fit both the linear model and the degree-3 polynomial model on the training data, then predict MPG on the validation fold and compute the mean squared error (MSE). The MSEs from the 10 folds are averaged to produce one cross-validation error for that repetition. Repeating the procedure 50 times gives 50 estimates of the mean CV MSE for each model. I then compute the average of those values, their standard deviation, and the training MSE using the full dataset so the models can be compared using both validation and training error.

```r
rep_mean_cv_linear <- numeric(n_rep)
rep_mean_cv_poly <- numeric(n_rep)

for (r in seq_len(n_rep)) {
  fold_id <- sample(rep(seq_len(K), length.out = n))
  stopifnot(length(fold_id) == n)

  mse_linear <- numeric(K)
  mse_poly <- numeric(K)

  for (k in seq_len(K)) {
    train <- fold_id != k
    val <- fold_id == k

    fit_lin <- lm(mpg ~ horsepower, data = Auto, subset = train)
    fit_poly <- lm(
      mpg ~ poly(horsepower, degree = poly_degree, raw = TRUE),
      data = Auto,
      subset = train
    )

    pred_lin <- predict(fit_lin, newdata = Auto[val, , drop = FALSE])
    pred_poly <- predict(fit_poly, newdata = Auto[val, , drop = FALSE])

    y_val <- Auto$mpg[val]
    mse_linear[k] <- mean((y_val - pred_lin)^2)
    mse_poly[k] <- mean((y_val - pred_poly)^2)
  }

  rep_mean_cv_linear[r] <- mean(mse_linear)
  rep_mean_cv_poly[r] <- mean(mse_poly)
}

mean_cv_mse_linear <- mean(rep_mean_cv_linear)
mean_cv_mse_poly <- mean(rep_mean_cv_poly)
sd_rep_linear <- sd(rep_mean_cv_linear)
sd_rep_poly <- sd(rep_mean_cv_poly)
mean_cv_mse_linear
```

```
[1] 24.26072
```

```r
mean_cv_mse_poly
```

```
[1] 19.35143
```

```r
sd_rep_linear
```

```
[1] 0.0986
```

```r
sd_rep_poly
```

```
[1] 0.1112483
```

```r
fit_lin_full <- lm(mpg ~ horsepower, data = Auto)
fit_poly_full <- lm(
  mpg ~ poly(horsepower, degree = poly_degree, raw = TRUE),
  data = Auto
)

train_mse_linear <- mean(residuals(fit_lin_full)^2)
train_mse_poly <- mean(residuals(fit_poly_full)^2)
train_mse_linear
```

```
[1] 23.94366
```

```r
train_mse_poly
```

```
[1] 18.94499
```

Results

Table 1: Each row is one full 10-fold CV run: mean validation MSE for that run (same folds for linear vs polynomial within the row).

repetition	mean_CV_MSE_linear	mean_CV_MSE_polynomial
1	24.3081	19.2771
2	24.1914	19.4476

repetition	mean_CV_MSE_linear	mean_CV_MSE_polynomial
3	24.2876	19.3533
4	24.2057	19.2513
5	24.3817	19.4036
6	24.1983	19.3926
7	24.2775	19.2874
8	24.2542	19.2626
9	24.3141	19.6074
10	24.1668	19.2866
11	24.3414	19.4068
12	24.2247	19.2879
13	24.1589	19.3501
14	24.2293	19.2814
15	24.1781	19.3056
16	24.0836	19.1614
17	24.0935	19.1218
18	24.2251	19.3985
19	24.3645	19.2942
20	24.1256	19.2791
21	24.2691	19.2582
22	24.1856	19.2582
23	24.4370	19.5675
24	24.1975	19.4424
25	24.2620	19.5216
26	24.3273	19.3484
27	24.4261	19.2900
28	24.2751	19.2041
29	24.1931	19.1892
30	24.2596	19.4754
31	24.0986	19.4769
32	24.2475	19.3234
33	24.1753	19.2109
34	24.2937	19.4815
35	24.2247	19.2117
36	24.5677	19.3515
37	24.3463	19.4154
38	24.2203	19.2385
39	24.3240	19.3590
40	24.2677	19.2852
41	24.3562	19.5582
42	24.2915	19.4925
43	24.2906	19.3615

repetition	mean_CV_MSE_linear	mean_CV_MSE_polynomial
44	24.3067	19.3826
45	24.0587	19.5441
46	24.4417	19.4850
47	24.2234	19.3473
48	24.3502	19.3409
49	24.2901	19.3311
50	24.2188	19.3625

Table 2: Across 50 repeated 10-fold CVs: average of the 50 run-level mean CV MSEs, SD across those 50 values, and training MSE on full data.

model	mean_of_repetition_means	sd_across_repetitions	training_MSE_full_data
Linear	24.2607	0.0986	23.9437
Polynomial (degree 3)	19.3514	0.1112	18.9450

Averaging across the **50** repeated 10-fold CV runs, the mean CV MSE is **24.2607** for the linear model and **19.3514** for the polynomial model. Since the polynomial model has the smaller validation error, I would choose the **polynomial (degree 3) model: mpg ~ poly(horsepower, 3)**.

```
x_seq <- seq(min(Auto$horsepower), max(Auto$horsepower), length.out = 200)
newdf <- data.frame(horsepower = x_seq)
pred_line <- predict(fit_lin_full, newdata = newdf)
pred_curve <- predict(fit_poly_full, newdata = newdf)

plot(
  Auto$horsepower,
  Auto$mpg,
  xlab = "Horsepower",
  ylab = "MPG",
  main = "MPG vs horsepower (full-data fits for intuition)"
)
lines(x_seq, pred_line, col = "blue", lwd = 2)
lines(x_seq, pred_curve, col = "red", lwd = 2)
legend(
  "topright",
  legend = c("Linear", paste0("Polynomial deg. ", poly_degree)),
```

```
  col = c("blue", "red"),
  lwd = 2
)
```

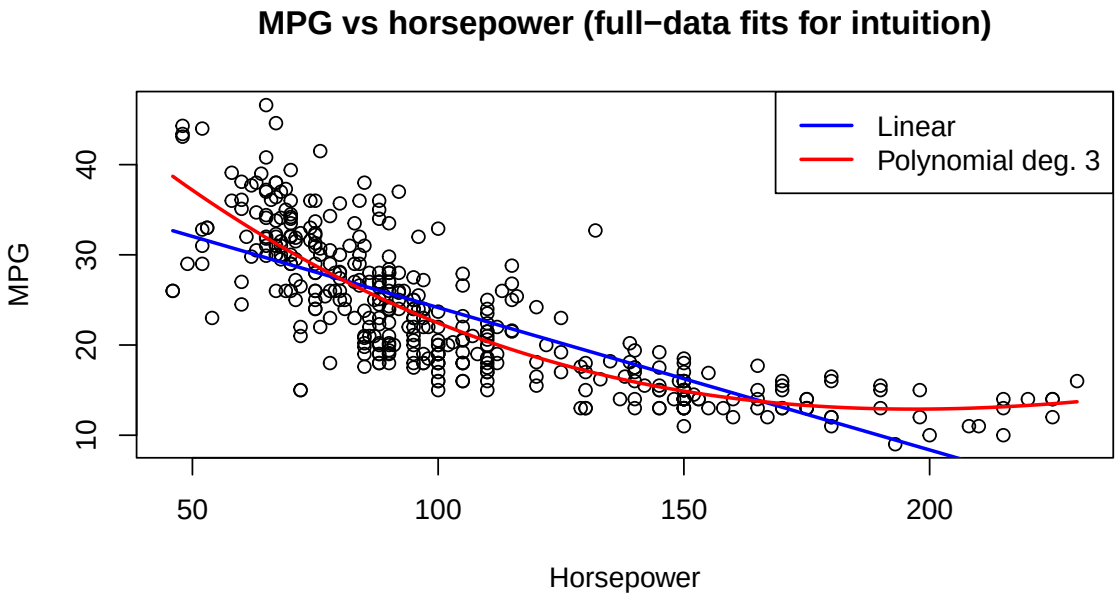

Figure 1: Quick visual: I fit both models on all the data just to show the shape of the fits. The actual model choice above comes from cross-validation, not from how nice this plot looks.

Conclusion

In this project, I compared a simple linear model and a cubic polynomial model for predicting MPG using horsepower from the Auto dataset. To make the comparison fair, I used 10-fold cross-validation repeated 50 times, with both models evaluated on the same folds each time.

Looking at the average validation error across the 50 runs, the polynomial model performed noticeably better. The mean CV MSE was about 24.26 for the linear model and 19.35 for the polynomial model, so the polynomial model consistently had lower prediction error. Because cross-validation measures how well a model performs on unseen data, this result suggests that the cubic polynomial captures the relationship between horsepower and MPG more effectively than a straight line.

Training error alone could be misleading here. As expected, the polynomial model also had a lower training MSE, since more flexible models tend to fit the existing data better. That is exactly why cross-validation is important. It helps evaluate how the model is likely to perform on new observations rather than just how well it fits the current dataset.

Overall, based on the repeated cross-validation results, I would choose the degree-3 polynomial model for predicting MPG from horsepower in this dataset. However, it's important to remember that this analysis only focuses on prediction with one variable. Since the data are observational, this model should not be interpreted as evidence that horsepower directly causes changes in MPG.